PROJECT REPORT

Programming Fundamental (DS-121L)

1st Semester

TOPIC: Library Management System

Group Members:

Zainab Jameel (2025F-BDS-013)

Anosha Shahid (2025F-BDS-005)

Muhammad Bazil (2025F-BDS-017)

Shaikh Taha (2025F-BDS-025)

Submitted to: Sir Habib Shaukat

AC&ES Department

Data Science Program

Table of Contents

1. Acknowledgment 2
2. Abstract 2
3. Background 2
4. Objectives 2
5. Scope 3
6. Implementation of Data Structures 3
   1. Dictionaries 3
   2. Lists & Tuples 3
   3. Relational Database (Table) 3
7. Source Code 3
   loginPage.py: 3
   lmsPage.py: 6
   books.py: 11
   database.py: 12
8. Output 14
   Login Screen 14
   Main Dashboard 14
   Validation Warnings 15
9. Explanation 16
10. System Design 16
11. Features 17
12. Classes & Methods 17
13. User Interface 18
14. Code Flow 18
15. Technologies Used 18
16. Future Enhancements 19
17. Conclusion 19

1. Acknowledgment

We would like to extend our sincere thanks to our supervisor, mentors, and colleagues for their unwavering support and encouragement throughout the course of this project. Special recognition goes to Sir Syed University of Engineering and Technology for providing an enriching environment for academic growth and offering the necessary tools and resources to complete this project.

2. Abstract

The Library Management System (LMS) project provides a digital platform to manage the issuing and returning of books efficiently. This is achieved through a user-friendly GUI built using Python's Tkinter library. The project bridges the gap between manual record-keeping and automated database management. By using a local SQLite database, the system ensures data persistence, allowing users to track inventory in real time.This tool is ideal for educational institutions looking to streamline their library operations.

3. Background

Library management is a fundamental aspect of educational administration. Traditionally, libraries used manual card systems to track borrowed books, which was time-consuming and prone to human error. Understanding how to automate these processes is essential for modern data science and software development. The LMS project addresses this by providing a graphical interface where students can log in using their valid identification and manage book transactions seamlessly.

4. Objectives

The objectives of the Library Management System are as follows:

1. Automation: To automate the manual process of issuing and returning books.
2. Data Persistence: To store records of issued books securely using a relational database (SQLite).
3. Validation: To implement input validation (Regex) ensuring only valid student IDs and phone numbers are accepted.
4. Inventory Management: To provide real-time updates on book stock availability.

5. Scope

The project aims to serve the following purposes:
1. Educational Tool: Targeted at university students and staff to manage library resources.
2. Domain Specificity: The current book inventory focuses on Data Science and Machine Learning resources.
3. Visual Interface: The scope includes a fully functional GUI with login verification and a dashboard for library transactions.
4. Future Expansion: The project can be expanded to include faculty logins, fine calculation for late returns, and cloud database integration.

6. Implementation of Data Structures

This section outlines the data structures used in the LMS project:

1. Dictionaries

- Description: Used to store the static inventory of books and their stock counts.
- Implementation: The variable 'data_science_books' in 'books.py' maps book titles (Keys) to their available quantity (Values).

2. Lists & Tuples

- Description: Used to handle data retrieved from the database.
- Implementation: When 'database.get_books' is called, it returns a list of tuples, where each tuple represents a row (ID, Title, Date, Time) from the 'issued_books' table.

3. Relational Database (Table)

- Description: An SQLite table named 'issued_books' acts as the persistent structure.
- Schema: Stores id, student_id, name, email, phone, book, date, and time.

7. Source Code

loginPage.py:

```
import tkinter as tk
from tkinter import messagebox
from PIL import Image, ImageTk
import os
import lmsPage
import re

COLORS = {
```

```
    "bg_card": "#FFFFFF",
    "primary": "#2C3E50",
    "accent": "#3498DB",
    "accent_hover": "#2980B9",
    "text": "#34495E",
}


def verify_login():
    student_id = entry_id.get().strip().upper()  # convert input to uppercase

    if student_id == "":
        messagebox.showwarning("Input Required", "Please write your student ID.")
        return

    # Pattern: 20YY[F or S]-XXX-NNN
    pattern = r"^20\d{2}[FS]-[A-Z]{3,7}-\d{3,5}$"

    if not re.match(pattern, student_id):
        messagebox.showerror(
            "Invalid ID Format",
            "Student ID must be in this format:\n\n2025F-BDS-017\nor 2025S-BDS-025\n(All capital letters)"
        )
        return

    # Login successful
    messagebox.showinfo("Login Successful", f"Welcome, Student {student_id}!")
    root.withdraw()
    lmsPage.open_lms_window(root, student_id)


root = tk.Tk()
root.title("Student Login")
root.geometry("1000x860")

canvas = tk.Canvas(root, highlightthickness=0, bg=COLORS["primary"])
canvas.pack(fill="both", expand=True)

image_path = "background.jpg"
original_image = None
bg_item = None

if os.path.exists(image_path):
    try:
        original_image = Image.open(image_path)
        bg_image = ImageTk.PhotoImage(original_image.resize((1, 1)))
        bg_item = canvas.create_image(0, 0, image=bg_image, anchor="nw")
    except Exception as e:
        print(f"Image load error: {e}")
```

```
card_frame = tk.Frame(root, bg=COLORS["bg_card"], bd=0)

header_stripe = tk.Frame(card_frame, bg=COLORS["primary"], height=80)
header_stripe.pack(fill="x")
header_stripe.pack_propagate(False)

tk.Label(header_stripe, text="LMS PORTAL", font=("Helvetica", 20, "bold"),
        bg=COLORS["primary"], fg="white").pack(expand=True)

form_frame = tk.Frame(card_frame, bg=COLORS["bg_card"])
form_frame.pack(pady=40, padx=40, fill="x")

tk.Label(form_frame, text="Write your student id", font=("Helvetica", 11, "bold"),
        bg=COLORS["bg_card"], fg=COLORS["text"]).pack(anchor="w", pady=(0, 5))

entry_id = tk.Entry(form_frame, font=("Helvetica", 12), relief="flat", bg="#F0F3F4", justify="center")
entry_id.pack(fill="x", ipady=8)
tk.Frame(form_frame, bg=COLORS["accent"], height=2).pack(fill="x")

def on_enter(e):
    btn_submit['background'] = COLORS["accent_hover"]

def on_leave(e):
    btn_submit['background'] = COLORS["accent"]

btn_submit = tk.Button(card_frame, text="LOGIN", font=("Helvetica", 12, "bold"),
                    bg=COLORS["accent"], fg="white",
                    activebackground=COLORS["accent_hover"], activeforeground="white",
                    relief="flat", cursor="hand2", command=verify_login)
btn_submit.bind("<Enter>", on_enter)
btn_submit.bind("<Leave>", on_leave)
btn_submit.pack(pady=20, ipadx=40, ipady=8)

def resize_ui(event):
    w = event.width
    h = event.height
    if original_image and bg_item:
        try:
            img_w, img_h = original_image.size
            ratio = max(w / img_w, h / img_h)
            new_size = (int(img_w * ratio), int(img_h * ratio))
            resized_image = original_image.resize(new_size, Image.Resampling.LANCZOS)
            new_photo = ImageTk.PhotoImage(resized_image)
            canvas.itemconfig(bg_item, image=new_photo)
            canvas.image = new_photo
            canvas.coords(bg_item, (w - new_size[0]) // 2, (h - new_size[1]) // 2)
        except:
            pass
```

```
    canvas.create_window(w // 2, h // 2, window=card_frame, width=380, height=420)

canvas.bind("<Configure>", resize_ui)

root.mainloop()
```

lmsPage.py:

```
import database
import tkinter as tk
from tkinter import ttk, messagebox
import books
from books import data_science_books
import re

# Colors
COLORS = {
    "bg_main": "#E8ECF4",
    "bg_card": "#FFFFFF",
    "primary": "#2C3E50",
    "accent": "#3498DB",
    "accent_hover": "#2980B9",
    "danger": "#E74C3C",
    "danger_hover": "#C0392B",
    "neutral": "#95A5A6",
    "neutral_hover": "#7F8C8D",
    "text": "#34495E",
    "light_text": "#7F8C8D"
}

# Static Data
STATIC_BOOKS = data_science_books


def create_button(parent, text, bg_color, hover_color, command=None):
    btn = tk.Button(parent, text=text, font=("Helvetica", 10, "bold"), bg=bg_color, fg="white",
                    activebackground=hover_color, activeforeground="white",
                    relief="flat", cursor="hand2", borderwidth=0, command=command)
    btn.bind("<Enter>", lambda e: btn.config(bg=hover_color))
    btn.bind("<Leave>", lambda e: btn.config(bg=bg_color))
    return btn


def open_lms_window(login_window, student_id):
    database.create_table()
    lms_window = tk.Toplevel()
    lms_window.title(f"Library Management System - {student_id}")
    lms_window.configure(bg=COLORS["bg_main"])
```

```
# --- DYNAMIC GEOMETRY ---
screen_height = lms_window.winfo_screenheight()
app_height = min(880, int(screen_height * 0.9))
x_c = int((lms_window.winfo_screenwidth() / 2) - (550 / 2))
y_c = int((screen_height / 2) - (app_height / 2))
lms_window.geometry(f"550x{app_height}+{x_c}+{y_c}")

def on_close():
    login_window.destroy()

lms_window.protocol("WM_DELETE_WINDOW", on_close)

def logout():
    confirm = messagebox.askyesno("Logout", "Are you sure you want to log out?")
    if confirm:
        lms_window.destroy()
        login_window.deiconify()

# 1. Header
header_frame = tk.Frame(lms_window, bg=COLORS["primary"], height=80)
header_frame.pack(fill="x", side="top")
header_frame.pack_propagate(False)

tk.Label(header_frame, text="LIBRARY MANAGER", font=("Helvetica", 20, "bold"),
         bg=COLORS["primary"], fg="white").pack(pady=25)

# 2. Main Card Frame
card_frame = tk.Frame(lms_window, bg=COLORS["bg_card"], bd=0, highlightthickness=0)
card_frame.pack(pady=20, padx=20, fill="both", expand=True)

# 3. Bottom Frame
bottom_frame = tk.Frame(card_frame, bg=COLORS["bg_card"])
bottom_frame.pack(side="bottom", fill="x", pady=20)

# --- Helper Functions ---
def refresh_stock_label():
    """Updates the stock label based on the currently selected dropdown item."""
    current_book = book_dropdown.get()
    if current_book in STATIC_BOOKS:
        qty_label.config(text=f"In Stock: {STATIC_BOOKS[current_book]}")
    else:
        qty_label.config(text="In Stock: -")

def load_books():
    books_listbox.delete(0, tk.END)
    books_data = database.get_books(student_id)

    # database.get_books returns: (id, book_title, date, time)
```

```
        # Index 0 = ID
        # Index 1 = Book Title
        # Index 2 = Date

        for row in books_data:
            book_title = row[1]

            # Format: ID | Book Name | Date
            books_listbox.insert(tk.END, f"{row[0]} | {book_title} | {row[2]}")

    def return_book_action():
        selected = books_listbox.curselection()
        if not selected:
            messagebox.showwarning("Error", "Please select a book to return")
            return

        list_item = books_listbox.get(selected[0])
        parts = list_item.split(" | ")

        if len(parts) >= 2:
            book_id = parts[0].strip()
            book_title = parts[1].strip()

            # 1. Update Database
            database.delete_book(book_id)

            # 2. Update Stock
            if book_title in STATIC_BOOKS:
                STATIC_BOOKS[book_title] += 1

                if book_dropdown.get() == book_title:
                    refresh_stock_label()
            else:
                print(f"DEBUG: Could not find '{book_title}' in list.")

            load_books()
        else:
            messagebox.showerror("Error", "Could not parse book data.")

    # --- Bottom Buttons ---
    btn_remove = create_button(
        bottom_frame, "RETURN SELECTED", COLORS["danger"], COLORS["danger_hover"],
        command=return_book_action
    )
    btn_remove.pack(pady=(0, 10), ipadx=20, ipady=5, anchor="center")

    btn_logout = create_button(bottom_frame, "LOGOUT", COLORS["neutral"],
COLORS["neutral_hover"], command=logout)
    btn_logout.pack(ipadx=40, ipady=5, anchor="center")
```

```
# --- Middle Content ---
content_container = tk.Frame(card_frame, bg=COLORS["bg_card"])
content_container.pack(side="top", fill="both", expand=True)

form_frame = tk.Frame(content_container, bg=COLORS["bg_card"])
form_frame.pack(pady=10)

labels = ["Student Name", "Student Email", "Phone Number"]
entries = {}

for i, label_text in enumerate(labels):
    tk.Label(form_frame, text=label_text, font=("Helvetica", 10, "bold"), bg=COLORS["bg_card"],
             fg=COLORS["text"]).grid(row=i * 2, column=0, sticky="w", pady=(0, 5))
    entry = tk.Entry(form_frame, width=30, font=("Helvetica", 11), relief="flat", bg="#F0F3F4")
    entry.grid(row=i * 2 + 1, column=0, ipady=5, padx=5, pady=(0, 15))
    entries[label_text] = entry

tk.Label(form_frame, text="Select Book", font=("Helvetica", 10, "bold"), bg=COLORS["bg_card"],
         fg=COLORS["text"]).grid(row=6, column=0, sticky="w", pady=(0, 5))

style = ttk.Style()
style.theme_use('clam')
style.configure("TCombobox", fieldbackground="#F0F3F4", background="white", arrowsize=15)

book_dropdown = ttk.Combobox(form_frame, width=28, font=("Helvetica", 11), state="readonly")
book_dropdown['values'] = list(STATIC_BOOKS.keys())
book_dropdown.set("Select a Book")
book_dropdown.grid(row=7, column=0, ipady=5, padx=5)

qty_label = tk.Label(form_frame, text="In Stock: -", font=("Helvetica", 9), bg=COLORS["bg_card"],
                     fg=COLORS["light_text"])
qty_label.grid(row=8, column=0, sticky="e", pady=5)

def on_combobox_select(event):
    refresh_stock_label()

book_dropdown.bind("<<ComboboxSelected>>", on_combobox_select)

# --- Validation ---
def validate_phone(phone_str):
    return phone_str.isdigit() and 10 <= len(phone_str) <= 15

def validate_email(email_str):
    pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    return re.match(pattern, email_str) is not None

def issue_book_action():
    name = entries["Student Name"].get().strip()
```

```
        email = entries["Student Email"].get().strip()
        phone = entries["Phone Number"].get().strip()
        book = book_dropdown.get()

        if not name or not email or not phone or book == "Select a Book":
            messagebox.showwarning("Missing Info", "Please fill all fields and select a book.")
            return

        if not validate_phone(phone):
            messagebox.showerror("Invalid Phone", "Phone number must be digits only (10-15 chars).")
            return

        if not validate_email(email):
            messagebox.showerror("Invalid Email", "Please enter a valid email address.")
            return

        # Stock Check
        current_stock = STATIC_BOOKS.get(book, 0)

        if current_stock <= 0:
            messagebox.showerror("Out of Stock", f"Sorry, '{book}' is currently out of stock.")
            return

        # Issue Book
        database.issue_book(student_id, name, email, phone, book)

        # Decrease Stock
        STATIC_BOOKS[book] -= 1
        refresh_stock_label()

        messagebox.showinfo("Success", "Book issued successfully")



        load_books()

    btn_add = create_button(
        content_container, "ISSUE BOOK", COLORS["accent"], COLORS["accent_hover"],
        command=issue_book_action
    )
    btn_add.pack(pady=5, ipadx=40, ipady=5)

    ttk.Separator(content_container, orient="horizontal").pack(fill="x", padx=20, pady=10)

    tk.Label(content_container, text="Currently Borrowed", font=("Helvetica", 10, "bold"),
bg=COLORS["bg_card"],
             fg=COLORS["text"]).pack(pady=5)

    list_frame = tk.Frame(content_container, bg=COLORS["bg_card"])
```

```
    list_frame.pack(padx=20, pady=5, fill="both", expand=True)

    books_listbox = tk.Listbox(list_frame, width=40, height=5, font=("Courier", 10),
                               bg="#F9FAFB", fg=COLORS["text"], bd=0, highlightthickness=0,
                               selectbackground=COLORS["accent"])
    books_listbox.pack(side="left", fill="both", expand=True)

    scrollbar = ttk.Scrollbar(list_frame, orient="vertical", command=books_listbox.yview)
    scrollbar.pack(side="right", fill="y")
    books_listbox.config(yscrollcommand=scrollbar.set)

    load_books()
```

books.py:

```
data_science_books = {
    "Python for Data Science": 15,
    "Machine Learning Fundamentals": 8,
    "Deep Learning with Neural Networks": 12,
    "Statistics for Data Analysis": 20,
    "Data Visualization Essentials": 9,
    "Pandas and NumPy Mastery": 14,
    "Applied Data Science": 6,
    "Big Data Analytics": 18,
    "Data Mining Concepts": 10,
    "AI for Data Scientists": 7,

    "Regression Analysis Explained": 16,
    "Time Series Forecasting": 11,
    "Natural Language Processing Basics": 5,
    "Computer Vision Foundations": 13,
    "Exploratory Data Analysis": 17,
    "Data Cleaning Techniques": 8,
    "Feature Engineering Guide": 19,
    "Predictive Modeling": 9,
    "Probability for Data Science": 14,
    "SQL for Data Analysts": 6,

    "R for Data Science": 15,
    "Bayesian Statistics Made Easy": 7,
    "Dimensionality Reduction": 12,
    "Clustering Algorithms": 18,
    "Recommendation Systems": 10,
    "Decision Trees and Random Forests": 5,
    "Support Vector Machines": 16,
    "Neural Network Architectures": 11,
    "Optimization Methods": 9,
    "Data Ethics and Privacy": 20,
```

```
    "Cloud Computing for Data Science": 8,
    "Spark for Big Data": 14,
    "Hadoop Essentials": 6,
    "Streaming Data Processing": 17,
    "Data Warehousing Concepts": 10,
    "Business Intelligence Tools": 13,
    "Power BI for Analysts": 7,
    "Tableau Data Visualization": 15,
    "Dashboards and Reporting": 9,
    "Kaggle Projects Guide": 18,

    "AutoML Techniques": 11,
    "Model Evaluation Metrics": 5,
    "A/B Testing for Data Science": 16,
    "Causal Inference": 12,
    "Reinforcement Learning Intro": 19,
    "Graph Analytics": 8,
    "Anomaly Detection": 14,
    "Data Science Capstone Projects": 6,
    "Real World Machine Learning": 17,
    "Explainable AI": 10,

    "Data Science with Excel": 9,
    "Mathematics for Machine Learning": 15,
    "Data Pipelines and ETL": 7,
    "NoSQL Databases for Data Science": 13,
    "Data Science in Healthcare": 18,
    "Financial Data Analysis": 11,
    "Marketing Analytics": 5,
    "Social Media Data Mining": 16,
    "Geospatial Data Analysis": 12,
    "Data Science Interview Prep": 20,

    "Edge AI and IoT Analytics": 8,
    "Time Series with Python": 14,
    "Advanced SQL Analytics": 6,
    "Text Mining Techniques": 17,
    "Image Processing with Python": 10,
    "Statistical Learning Theory": 9,
    "Data Science in Education": 15,
    "Energy Data Analytics": 7,
    "Sports Analytics": 13,
    "Smart Cities Data Science": 18
}
```

database.py:

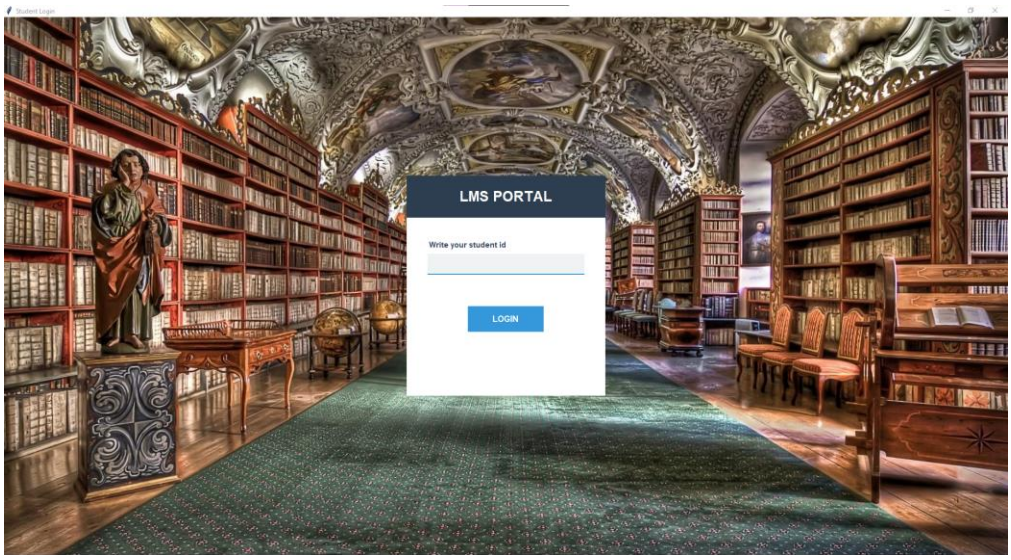
```
import sqlite3
```

```
from datetime import datetime

def connect_db():
    return sqlite3.connect("lms.db")

def create_table():
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS issued_books (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id TEXT,
            name TEXT,
            email TEXT,
            phone TEXT,
            book TEXT,
            date TEXT,
            time TEXT
        )
    """)
    conn.commit()
    conn.close()

def issue_book(student_id, name, email, phone, book):
    conn = connect_db()
    cur = conn.cursor()

    now = datetime.now()
    date = now.strftime("%Y-%m-%d")
    time = now.strftime("%H:%M:%S")

    cur.execute("""
        INSERT INTO issued_books
        VALUES (NULL, ?, ?, ?, ?, ?, ?, ?)
    """, (student_id, name, email, phone, book, date, time))

    conn.commit()
    conn.close()

def get_books(student_id):
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("""
        SELECT id, book, date, time
        FROM issued_books
        WHERE student_id = ?
    """, (student_id,))
    rows = cur.fetchall()
    conn.close()
```

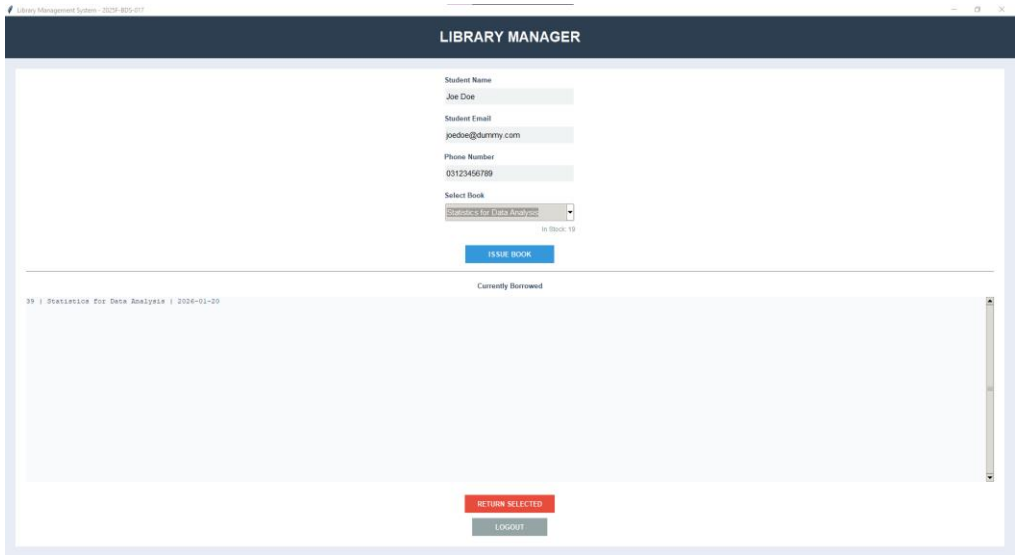
```
    return rows

def delete_book(book_id):
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("DELETE FROM issued_books WHERE id = ?", (book_id,))
    conn.commit()
    conn.close()
```

8. Output

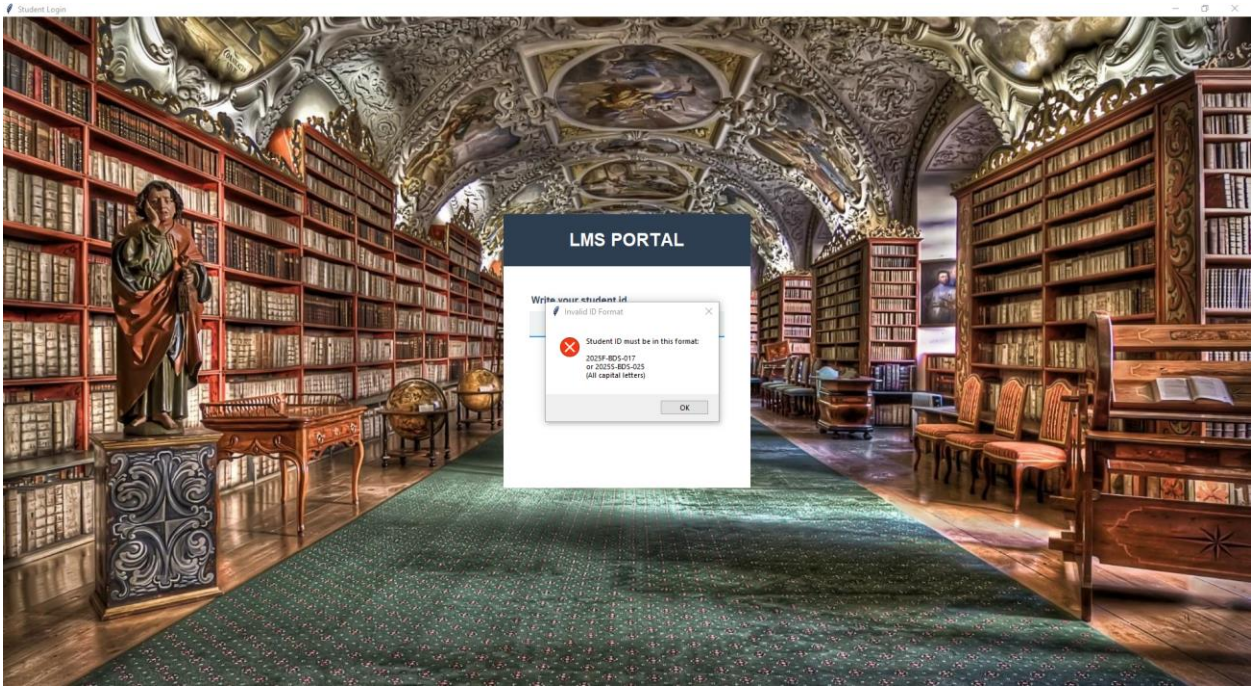
Login Screen

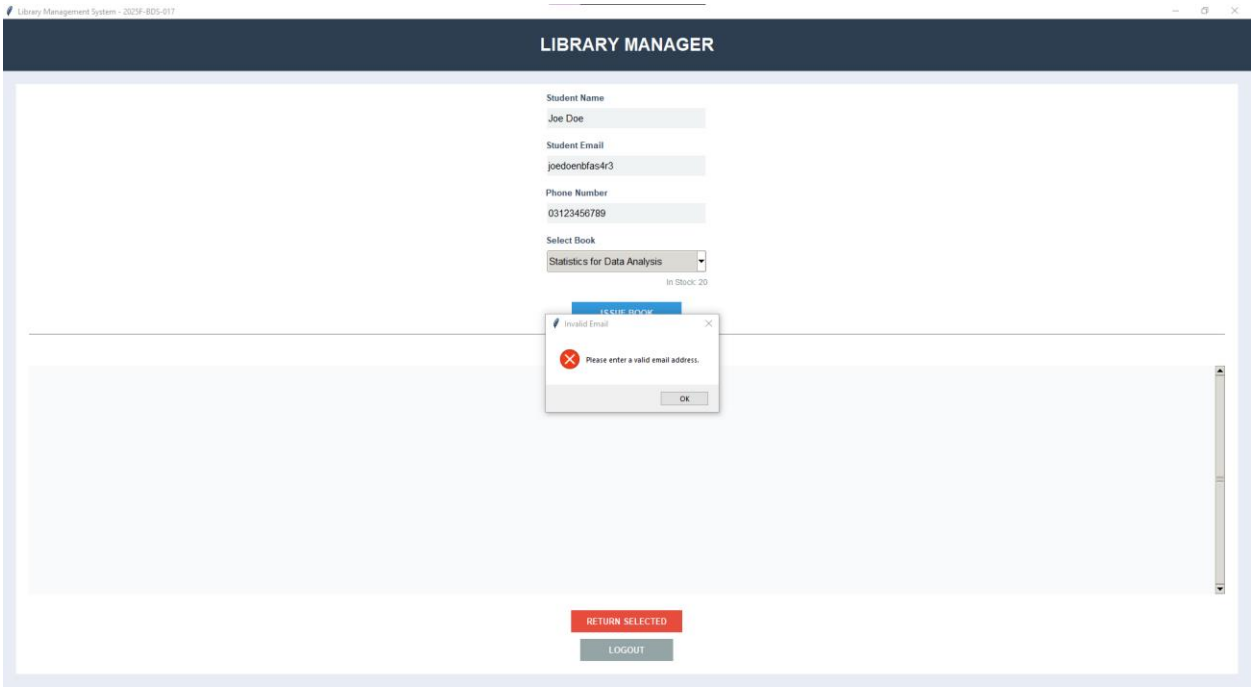


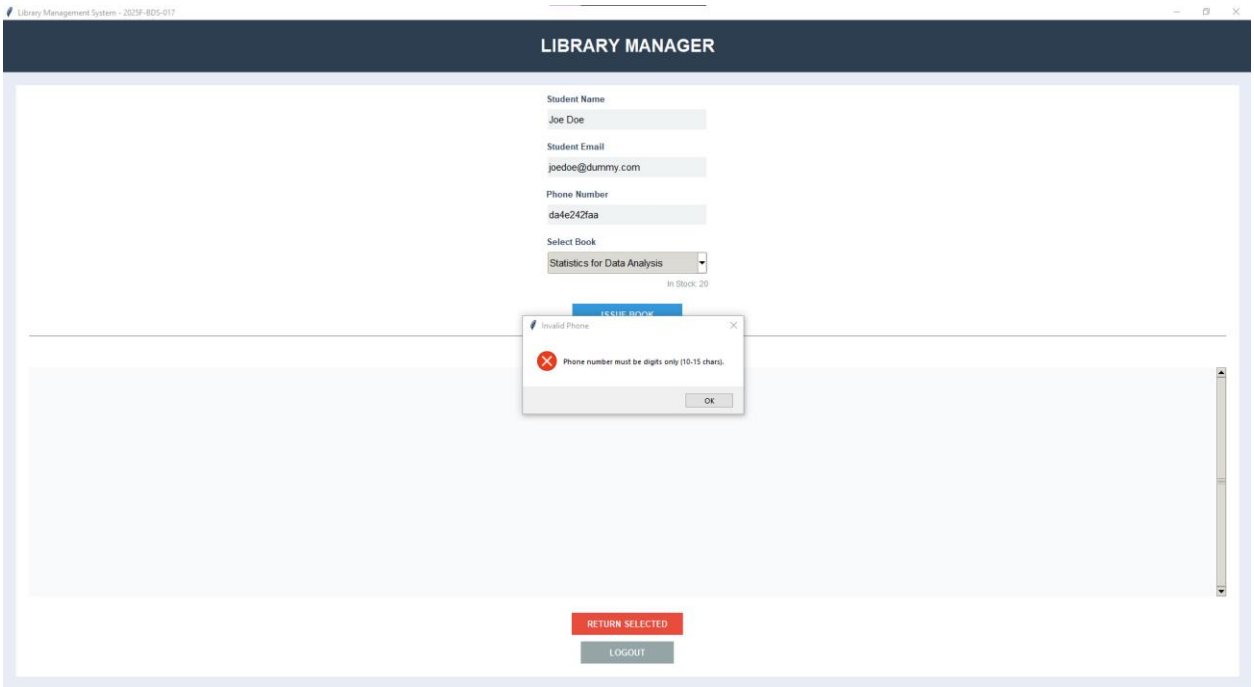
Main Dashboard



Validation Warnings







9. Explanation

The Library Management System works by first authenticating the user. The 'loginPage.py' script validates the student ID format using Regular Expressions (Regex) to ensure it matches the pattern '20YY[F or S]-XXX-NNN'.

Once logged in, the 'lmsPage.py' window opens. Here, the system connects to 'lms.db' to retrieve any books previously issued to that specific student . Users can select a book from the dropdown menu (populated from 'books.py'), enter their contact details, and issue the book. The system automatically updates the local stock count and saves the transaction timestamp to the database

10. System Design

The system is designed as follows:

1. Frontend (GUI): Created using Tkinter. It utilizes Frames for layout management, Entry widgets for input, and Listboxes for displaying data.
2. Backend (Logic): Python functions handle event processing (button clicks, dropdown selection).
3. Database: SQLite3 is used for the backend storage, requiring no separate server installation.
4. Assets: Uses 'background.jpg' and PIL (Pillow) library for dynamic image

resizing on the login screen.

11. Features

1. Secure Login: Uses Regex '^20\d{2}[FS]-[A-Z]{3,7}-\d{3,5}$' to enforce strict ID formats.
2. Real-Time Stock Checking: The "In Stock" label updates immediately when a user selects a book from the dropdown.
3. Duplicate Prevention: Checks stock levels before issuing to prevent negative inventory.
4. Persistent History: Users can see a list of books they have borrowed even after closing and reopening the app.
5. Data Validation: Ensures phone numbers are digits only and emails follow standard formats.

12. Classes & Methods

Although the project is implemented using a Modular Procedural approach in Python, the logical structure maps to the following Classes and Methods:

A. CLASSES (MODULES)

1. loginPage:
   Acts as the main class for the application entry point. It handles the initialization of the application window, dynamic background rendering, and user authentication logic.
2. lmsPage:
   Represents the core Library Management System interface. This module manages the dashboard layout, event handling for issuing/returning books, and inventory updates.
3. database:
   Serves as the Data Access Object (DAO). It encapsulates all logic related to database connectivity, schema creation, and SQL query execution, ensuring separation of concerns.

B. METHODS

1. validate_email(email_str):
   [lmsPage] Uses Regex to verify that the student's email address follows standard formatting (user@domain.com).

2. validate_phone(phone_str):
   [lmsPage] Ensures the phone number contains only digits and falls within the valid length range (10-15 characters).
3. on_enter(e) / on_leave(e):
   [loginPage] UI Event methods that handle the hover effects for buttons, changing colors dynamically to improve user experience.
4. resize_ui(event):
   [loginPage] A dynamic method bound to the window configuration event. It recalculates image dimensions to ensure the background covers the screen responsively.

13. User Interface

The user interface is modern and intuitive:

1. Login Window: Features a responsive background image that resizes with the window.
2. LMS Dashboard: Divided into three sections:
   - Header: Displays "LIBRARY MANAGER".
   - Form Section: Inputs for Name, Email, Phone, and Book selection.
   - List Section: Displays currently borrowed books with a "Return" button

14. Code Flow

1. Start: Program initiates in 'loginPage.py'.
2. Input: User enters Student ID.
3. Validation: System checks ID format. If valid, the login window closes.
4. Transition: 'open_lms_window' is called, passing the 'student_id'.
5. Load: Database fetches existing records for that ID; UI loads book list.
6. Action: User issues a book -> Stock decreases -> Database updates.
7. End: User logs out or closes the application.

15. Technologies Used

1. Programming Language: Python 3
2. GUI Framework: Tkinter (Standard Python GUI library)
3. Database: SQLite3
4. Image Processing: PIL (Python Imaging Library / Pillow)

16. Future Enhancements

1. Admin Panel: A separate login for librarians to add/remove books from the inventory database.
2. Search Functionality: Adding a search bar to filter books by title or category.
3. Due Date Alerts: Sending email notifications when a book is due for return.
4. Barcode Integration: Scanning book ISBNs for faster checkout.

17. Conclusion

The Library Management System project successfully demonstrates the application of Python in building real-world utility software. By integrating a graphical interface with a relational database, the project highlights the importance of data persistence and user experience design. It serves as a robust foundation for learning software development, lifecycles and database management.